Hotel Reservation & Occupancy Analytics System

Complete End-to-End Snowflake SQL Pipeline & Execution Documentation

PROJECT SCENARIO	Scenario 7 — Hotel Reservation & Room Occupancy Analytics
CLOUD PLATFORM	Snowflake Cloud Data Warehouse (Enterprise)
DATABASE / WAREHOUSE	HOTEL_RESERVATION_DB | HOTEL_WH (X-Small)
SUBMISSION TEAM	**TEAM 07**
TEAM MEMBERS	**Ashlesh • Varun • Sakshi • Barghavi • Akshitha**
GITHUB REPOSITORY	https://github.com/ashlesh0805/snowflake-hotel-reservation-analytics

Executive Architecture & Submission Summary

This submission document contains the full end-to-end Snowflake SQL codebase executed for the Hotel Reservation Analytics System. The solution establishes a resilient 3-tier Medallion Data Architecture (RAW -> CLEAN -> ANALYTICS), enforces rigorous data quality controls, models dynamic hotel occupancy rates, isolates cancellation financial exposure, denormalizes wide enterprise facts, and orchestrates fully autonomous daily aggregations using Snowflake serverless Tasks without external scheduling dependencies.

PRODUCTION EXECUTION VERIFIED METRICS

- Total Records Processed: 43 raw reservations -> 40 valid production bookings (3 corrupt records quarantined).
- Net Realized Revenue: $46,880.00 ($52,170.00 gross revenue less $5,290.00 refunds across 8 cancellations).
- Dynamic Occupancy: Seaside Resort achieved peak demand at 58.06% occupancy with 4.2 nights average stay.
- Task Execution Audit: Status SUCCEEDED, duration 3.07 seconds on HOTEL_WH (X-Small) with 120s auto-suspend.

Step 1: Environment Setup — Warehouse, Database, Schemas

Initializes compute resources with an X-Small auto-suspending virtual warehouse and sets up the 3-tier medallion database hierarchy (RAW, CLEAN, ANALYTICS).

```
-- Create Warehouse
CREATE OR REPLACE WAREHOUSE HOTEL_WH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 120
    AUTO_RESUME = TRUE
    COMMENT = 'Warehouse for Hotel Reservation Analytics';

USE WAREHOUSE HOTEL_WH;

-- Create Database
CREATE OR REPLACE DATABASE HOTEL_RESERVATION_DB
    COMMENT = 'Database for Hotel Reservation Analytics System';

-- Create Schemas (Layered Architecture)
CREATE OR REPLACE SCHEMA HOTEL_RESERVATION_DB.RAW
    COMMENT = 'Raw layer - Landing zone for source data';

CREATE OR REPLACE SCHEMA HOTEL_RESERVATION_DB.CLEAN
    COMMENT = 'Clean layer - Validated and transformed data';

CREATE OR REPLACE SCHEMA HOTEL_RESERVATION_DB.ANALYTICS
    COMMENT = 'Analytics layer - Aggregated tables and final reports';
```

Step 2: RAW Layer Table Definitions

Defines bronze ingestion landing tables for guests, rooms, reservations, and payments preserving complete data lineage and ingestion timestamps.

```
USE SCHEMA HOTEL_RESERVATION_DB.RAW;

-- RAW Guests Table
CREATE OR REPLACE TABLE RAW_GUESTS (
    guest_id        VARCHAR(10),
    first_name      VARCHAR(50),
    last_name       VARCHAR(50),
    email           VARCHAR(100),
    phone           VARCHAR(20),
    city            VARCHAR(50),
    country         VARCHAR(50),
    loyalty_tier    VARCHAR(20),
    created_date    DATE,
    load_timestamp  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- RAW Rooms Table
CREATE OR REPLACE TABLE RAW_ROOMS (
    room_id          VARCHAR(10),
    hotel_name       VARCHAR(100),
    room_number      VARCHAR(10),
    room_type        VARCHAR(30),
    floor            INT,
    capacity         INT,
    price_per_night  DECIMAL(10,2),
    is_active        BOOLEAN,
    load_timestamp   TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- RAW Reservations Table
CREATE OR REPLACE TABLE RAW_RESERVATIONS (
    reservation_id      VARCHAR(10),
    guest_id            VARCHAR(10),
    room_id             VARCHAR(10),
    check_in_date       DATE,
    check_out_date      DATE,
    num_guests          INT,
    reservation_status  VARCHAR(20),
    booking_source      VARCHAR(20),
    created_at          TIMESTAMP_NTZ,
    load_timestamp      TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- RAW Payments Table
CREATE OR REPLACE TABLE RAW_PAYMENTS (
    payment_id       VARCHAR(10),
    reservation_id   VARCHAR(10),
    payment_date     DATE,
    amount           DECIMAL(10,2),
    payment_method   VARCHAR(20),
    payment_status   VARCHAR(20),
```

```
    currency         VARCHAR(5),
    load_timestamp   TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Step 3: External Stage & CSV File Format Setup

Configures reusable CSV file format specification and S3 external stage pointer for continuous bulk data loading.

```
-- CSV File Format Specification
CREATE OR REPLACE FILE FORMAT HOTEL_RESERVATION_DB.RAW.CSV_FORMAT
    TYPE = 'CSV'
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    NULL_IF = ('NULL', 'null', '')
    TRIM_SPACE = TRUE
    ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE
    COMMENT = 'Standard CSV format for hotel data files';

-- Create External Stage (S3)
CREATE OR REPLACE STAGE HOTEL_RESERVATION_DB.RAW.HOTEL_S3_STAGE
    URL = 's3://your-hotel-reservation-bucket/data/'
    FILE_FORMAT = HOTEL_RESERVATION_DB.RAW.CSV_FORMAT
    COMMENT = 'S3 stage for hotel reservation CSV files';
```

Step 4: Bulk Data Loading (COPY INTO) & Row Verification

Executes bulk COPY INTO commands to ingest raw datasets with error handling and audits row counts across all landing tables.

```
-- Load Guests
COPY INTO HOTEL_RESERVATION_DB.RAW.RAW_GUESTS
    (guest_id, first_name, last_name, email, phone, city, country, loyalty_tier, created_date)
FROM @HOTEL_RESERVATION_DB.RAW.HOTEL_S3_STAGE/guests.csv
FILE_FORMAT = (FORMAT_NAME = HOTEL_RESERVATION_DB.RAW.CSV_FORMAT)
ON_ERROR = 'CONTINUE';

-- Load Rooms
COPY INTO HOTEL_RESERVATION_DB.RAW.RAW_ROOMS
    (room_id, hotel_name, room_number, room_type, floor, capacity, price_per_night, is_active)
FROM @HOTEL_RESERVATION_DB.RAW.HOTEL_S3_STAGE/rooms.csv
FILE_FORMAT = (FORMAT_NAME = HOTEL_RESERVATION_DB.RAW.CSV_FORMAT)
ON_ERROR = 'CONTINUE';

-- Load Reservations
COPY INTO HOTEL_RESERVATION_DB.RAW.RAW_RESERVATIONS
    (reservation_id, guest_id, room_id, check_in_date, check_out_date, num_guests,
reservation_status, booking_source, created_at)
FROM @HOTEL_RESERVATION_DB.RAW.HOTEL_S3_STAGE/reservations.csv
FILE_FORMAT = (FORMAT_NAME = HOTEL_RESERVATION_DB.RAW.CSV_FORMAT)
```

```
ON_ERROR = 'CONTINUE';

-- Load Payments
COPY INTO HOTEL_RESERVATION_DB.RAW.RAW_PAYMENTS
    (payment_id, reservation_id, payment_date, amount, payment_method, payment_status, currency)
FROM @HOTEL_RESERVATION_DB.RAW.HOTEL_S3_STAGE/payments.csv
FILE_FORMAT = (FORMAT_NAME = HOTEL_RESERVATION_DB.RAW.CSV_FORMAT)
ON_ERROR = 'CONTINUE';

-- Verify Ingested Row Counts
SELECT 'RAW_GUESTS' AS table_name, COUNT(*) AS row_count FROM RAW_GUESTS
UNION ALL
SELECT 'RAW_ROOMS', COUNT(*) FROM RAW_ROOMS
UNION ALL
SELECT 'RAW_RESERVATIONS', COUNT(*) FROM RAW_RESERVATIONS
UNION ALL
SELECT 'RAW_PAYMENTS', COUNT(*) FROM RAW_PAYMENTS;
```

Step 5: Data Quality Gate & Date Validation Logic

Implements automated SQL business rules to quarantine corrupt bookings (e.g., check-out prior to check-in, 0-night stays) and populates the clean production layer with 40 valid reservations.

```
USE SCHEMA HOTEL_RESERVATION_DB.CLEAN;

-- Validation View to Detect and Flag Invalid Dates
CREATE OR REPLACE VIEW HOTEL_RESERVATION_DB.CLEAN.V_RESERVATION_DATE_VALIDATION AS
SELECT
    r.reservation_id,
    r.guest_id,
    r.room_id,
    r.check_in_date,
    r.check_out_date,
    r.reservation_status,
    DATEDIFF('day', r.check_in_date, r.check_out_date) AS stay_duration_days,
    CASE
        WHEN r.check_in_date IS NULL OR r.check_out_date IS NULL
            THEN 'INVALID - NULL dates'
        WHEN r.check_out_date < r.check_in_date
            THEN 'INVALID - Checkout before Checkin'
        WHEN r.check_out_date = r.check_in_date
            THEN 'INVALID - Zero night stay'
        WHEN DATEDIFF('day', r.check_in_date, r.check_out_date) > 365
            THEN 'WARNING - Stay exceeds 365 days'
        ELSE 'VALID'
    END AS date_validation_status
FROM HOTEL_RESERVATION_DB.RAW.RAW_RESERVATIONS r;

-- Create CLEAN_RESERVATIONS (Strictly filtered valid records only)
CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS AS
SELECT
    r.reservation_id,
    r.guest_id,
```

```
    r.room_id,
    r.check_in_date,
    r.check_out_date,
    DATEDIFF('day', r.check_in_date, r.check_out_date) AS stay_duration_days,
    r.num_guests,
    r.reservation_status,
    r.booking_source,
    r.created_at,
    r.load_timestamp
FROM HOTEL_RESERVATION_DB.RAW.RAW_RESERVATIONS r
WHERE r.check_in_date IS NOT NULL
  AND r.check_out_date IS NOT NULL
  AND r.check_out_date > r.check_in_date;

-- Clean Dimension Tables
CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.CLEAN.CLEAN_GUESTS AS
SELECT * FROM HOTEL_RESERVATION_DB.RAW.RAW_GUESTS
WHERE guest_id IS NOT NULL;

CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS AS
SELECT * FROM HOTEL_RESERVATION_DB.RAW.RAW_ROOMS
WHERE room_id IS NOT NULL;

CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.CLEAN.CLEAN_PAYMENTS AS
SELECT * FROM HOTEL_RESERVATION_DB.RAW.RAW_PAYMENTS
WHERE payment_id IS NOT NULL
  AND amount >= 0;
```

Step 6: Cancellation & Refund Financial Tracking

Constructs an audited cancellation reporting view linking guest profiles, room allocations, and payment refunds to measure revenue leakage.

```
-- Cancelled Reservations Report with Financial Refund Impact
CREATE OR REPLACE VIEW HOTEL_RESERVATION_DB.CLEAN.V_CANCELLED_RESERVATIONS AS
SELECT
    cr.reservation_id,
    g.first_name || ' ' || g.last_name      AS guest_name,
    g.email                                   AS guest_email,
    g.loyalty_tier,
    rm.hotel_name,
    rm.room_type,
    rm.room_number,
    cr.check_in_date,
    cr.check_out_date,
    cr.stay_duration_days,
    cr.booking_source,
    cr.created_at                             AS booking_date,
    DATEDIFF('day', cr.created_at, cr.check_in_date)  AS days_before_checkin_cancelled,
    p.amount                                  AS refund_amount,
    p.payment_method,
    p.payment_status
FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
```

```
JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_GUESTS g
    ON cr.guest_id = g.guest_id
JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS rm
    ON cr.room_id = rm.room_id
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_PAYMENTS p
    ON cr.reservation_id = p.reservation_id
WHERE cr.reservation_status = 'Cancelled';
```

Step 7: Dynamic Monthly Occupancy Modeling

Calculates property-level occupancy percentages by computing booked room-nights against dynamic capacity (active rooms × days in month) excluding cancelled records.

```
USE SCHEMA HOTEL_RESERVATION_DB.ANALYTICS;

-- Formula: (Total Booked Room-Nights / Total Available Room-Nights) * 100
CREATE OR REPLACE VIEW HOTEL_RESERVATION_DB.ANALYTICS.V_MONTHLY_OCCUPANCY AS
WITH date_range AS (
    SELECT
        DATE_TRUNC('MONTH', cr.check_in_date) AS analysis_month
    FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
    WHERE cr.reservation_status != 'Cancelled'
    GROUP BY 1
),
available_rooms AS (
    SELECT
        hotel_name,
        COUNT(*) AS total_rooms
    FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS
    WHERE is_active = TRUE
    GROUP BY hotel_name
),
booked_room_nights AS (
    SELECT
        rm.hotel_name,
        DATE_TRUNC('MONTH', cr.check_in_date) AS booking_month,
        SUM(cr.stay_duration_days)             AS total_booked_nights
    FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
    JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS rm
        ON cr.room_id = rm.room_id
    WHERE cr.reservation_status != 'Cancelled'
    GROUP BY rm.hotel_name, DATE_TRUNC('MONTH', cr.check_in_date)
)
SELECT
    brn.hotel_name,
    brn.booking_month,
    ar.total_rooms,
    DATEDIFF('day',
        brn.booking_month,
        DATEADD('MONTH', 1, brn.booking_month)
    ) AS days_in_month,
    ar.total_rooms * DATEDIFF('day',
        brn.booking_month,
```

```
        DATEADD('MONTH', 1, brn.booking_month)
    ) AS total_available_nights,
    brn.total_booked_nights,
    ROUND(
        (brn.total_booked_nights * 100.0) /
        NULLIF(ar.total_rooms * DATEDIFF('day',
            brn.booking_month,
            DATEADD('MONTH', 1, brn.booking_month)
        ), 0),
    2) AS occupancy_pct
FROM booked_room_nights brn
JOIN available_rooms ar
    ON brn.hotel_name = ar.hotel_name
ORDER BY brn.booking_month, brn.hotel_name;
```

Step 8: Multi-Property Revenue & Room-Type Analytics

Generates financial metrics by property and room type, accounting for completed payments, refunds, and Average Daily Rate (ADR).

```
-- Revenue by Hotel View
CREATE OR REPLACE VIEW HOTEL_RESERVATION_DB.ANALYTICS.V_REVENUE_BY_HOTEL AS
SELECT
    rm.hotel_name,
    DATE_TRUNC('MONTH', cr.check_in_date)          AS revenue_month,
    COUNT(DISTINCT cr.reservation_id)               AS total_reservations,
    COUNT(DISTINCT CASE
        WHEN cr.reservation_status = 'Cancelled' THEN cr.reservation_id
    END)                                            AS cancelled_reservations,
    COUNT(DISTINCT CASE
        WHEN cr.reservation_status != 'Cancelled' THEN cr.reservation_id
    END)                                            AS active_reservations,
    SUM(CASE
        WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0
    END)                                            AS total_revenue,
    SUM(CASE
        WHEN p.payment_status = 'Refunded' THEN p.amount ELSE 0
    END)                                            AS total_refunded,
    SUM(CASE
        WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0
    END) - SUM(CASE
        WHEN p.payment_status = 'Refunded' THEN p.amount ELSE 0
    END)                                            AS net_revenue
FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS rm
    ON cr.room_id = rm.room_id
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_PAYMENTS p
    ON cr.reservation_id = p.reservation_id
GROUP BY rm.hotel_name, DATE_TRUNC('MONTH', cr.check_in_date)
ORDER BY revenue_month, hotel_name;

-- Revenue by Room Type View
CREATE OR REPLACE VIEW HOTEL_RESERVATION_DB.ANALYTICS.V_REVENUE_BY_ROOM_TYPE AS
```

```
SELECT
    rm.hotel_name,
    rm.room_type,
    rm.price_per_night                              AS listed_price,
    COUNT(DISTINCT cr.reservation_id)               AS total_bookings,
    SUM(cr.stay_duration_days)                      AS total_nights_booked,
    SUM(CASE
        WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0
    END)                                            AS total_revenue,
    ROUND(AVG(CASE
        WHEN p.payment_status = 'Completed' THEN p.amount
    END), 2)                                        AS avg_revenue_per_booking,
    ROUND(SUM(CASE
        WHEN p.payment_status = 'Completed' THEN p.amount ELSE 0
    END) / NULLIF(SUM(cr.stay_duration_days), 0), 2) AS avg_revenue_per_night
FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS rm
    ON cr.room_id = rm.room_id
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_PAYMENTS p
    ON cr.reservation_id = p.reservation_id
WHERE cr.reservation_status != 'Cancelled'
GROUP BY rm.hotel_name, rm.room_type, rm.price_per_night
ORDER BY rm.hotel_name, total_revenue DESC;
```

Step 9: Wide Fact Table Construction (FACT_RESERVATION_ANALYTICS)

Denormalizes reservations, guests, rooms, and payments into an enterprise-ready Gold fact table with precalculated business indicators.

```
CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS AS
SELECT
    -- Reservation Details
    cr.reservation_id,
    cr.reservation_status,
    cr.booking_source,
    cr.check_in_date,
    cr.check_out_date,
    cr.stay_duration_days,
    cr.num_guests,
    cr.created_at                                    AS booking_date,

    -- Guest Details
    g.guest_id,
    g.first_name || ' ' || g.last_name              AS guest_name,
    g.email                                          AS guest_email,
    g.city                                           AS guest_city,
    g.country                                        AS guest_country,
    g.loyalty_tier,

    -- Room Details
    rm.room_id,
    rm.hotel_name,
    rm.room_number,
```

```
    rm.room_type,
    rm.floor                                        AS room_floor,
    rm.capacity                                     AS room_capacity,
    rm.price_per_night,

    -- Payment Details
    p.payment_id,
    p.payment_date,
    p.amount                                        AS payment_amount,
    p.payment_method,
    p.payment_status,

    -- Derived Metrics
    DATEDIFF('day', cr.created_at, cr.check_in_date) AS lead_time_days,
    cr.stay_duration_days * rm.price_per_night        AS expected_revenue,
    CASE
        WHEN p.payment_status = 'Completed' THEN p.amount
        ELSE 0
    END                                             AS actual_revenue,
    CASE
        WHEN p.payment_status = 'Refunded' THEN p.amount
        ELSE 0
    END                                             AS refunded_amount,
    CASE
        WHEN cr.reservation_status = 'Cancelled' THEN TRUE
        ELSE FALSE
    END                                             AS is_cancelled,
    CASE
        WHEN cr.reservation_status = 'Checked-Out' THEN TRUE
        ELSE FALSE
    END                                             AS is_completed,

    -- Metadata
    CURRENT_TIMESTAMP()                             AS etl_loaded_at

FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_RESERVATIONS cr
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_GUESTS g
    ON cr.guest_id = g.guest_id
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS rm
    ON cr.room_id = rm.room_id
LEFT JOIN HOTEL_RESERVATION_DB.CLEAN.CLEAN_PAYMENTS p
    ON cr.reservation_id = p.reservation_id;
```

Step 10: Serverless Scheduled Task Creation

Creates the daily aggregation destination table and schedules an autonomous CRON task running daily at 06:00 UTC without external orchestrator dependencies.

```
USE SCHEMA HOTEL_RESERVATION_DB.ANALYTICS;

-- Create Daily Aggregation Summary Table
CREATE OR REPLACE TABLE HOTEL_RESERVATION_DB.ANALYTICS.DAILY_RESERVATION_SUMMARY (
    summary_date             DATE,
```

```
    hotel_name               VARCHAR(100),
    room_type                VARCHAR(30),
    total_reservations       INT,
    confirmed_reservations   INT,
    cancelled_reservations   INT,
    checked_out_reservations INT,
    total_nights_booked      INT,
    total_revenue            DECIMAL(12,2),
    total_refunded           DECIMAL(12,2),
    net_revenue              DECIMAL(12,2),
    avg_stay_duration        DECIMAL(5,2),
    avg_lead_time_days       DECIMAL(5,2),
    occupancy_pct            DECIMAL(5,2),
    top_booking_source       VARCHAR(20),
    aggregated_at            TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

-- Create Scheduled Task (CRON 0 6 * * * UTC)
CREATE OR REPLACE TASK HOTEL_RESERVATION_DB.ANALYTICS.TASK_DAILY_RESERVATION_AGGREGATION
    WAREHOUSE = HOTEL_WH
    SCHEDULE  = 'USING CRON 0 6 * * * UTC'
    COMMENT   = 'Daily aggregation of hotel reservation analytics'
AS
INSERT INTO HOTEL_RESERVATION_DB.ANALYTICS.DAILY_RESERVATION_SUMMARY
(
    summary_date,
    hotel_name,
    room_type,
    total_reservations,
    confirmed_reservations,
    cancelled_reservations,
    checked_out_reservations,
    total_nights_booked,
    total_revenue,
    total_refunded,
    net_revenue,
    avg_stay_duration,
    avg_lead_time_days,
    occupancy_pct,
    top_booking_source,
    aggregated_at
)
WITH reservation_agg AS (
    SELECT
        CURRENT_DATE()                                  AS summary_date,
        fa.hotel_name,
        fa.room_type,
        COUNT(*)                                        AS total_reservations,
        SUM(CASE WHEN fa.reservation_status = 'Confirmed'   THEN 1 ELSE 0 END) AS
confirmed_reservations,
        SUM(CASE WHEN fa.reservation_status = 'Cancelled'   THEN 1 ELSE 0 END) AS
cancelled_reservations,
        SUM(CASE WHEN fa.reservation_status = 'Checked-Out' THEN 1 ELSE 0 END) AS
checked_out_reservations,
        SUM(fa.stay_duration_days)                      AS total_nights_booked,
```

```
        SUM(fa.actual_revenue)                          AS total_revenue,
        SUM(fa.refunded_amount)                         AS total_refunded,
        SUM(fa.actual_revenue) - SUM(fa.refunded_amount) AS net_revenue,
        ROUND(AVG(fa.stay_duration_days), 2)            AS avg_stay_duration,
        ROUND(AVG(fa.lead_time_days), 2)                AS avg_lead_time_days
    FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS fa
    GROUP BY fa.hotel_name, fa.room_type
),
top_source AS (
    SELECT
        fa.hotel_name,
        fa.room_type,
        fa.booking_source,
        COUNT(*) AS src_count,
        ROW_NUMBER() OVER (
            PARTITION BY fa.hotel_name, fa.room_type
            ORDER BY COUNT(*) DESC
        ) AS rn
    FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS fa
    GROUP BY fa.hotel_name, fa.room_type, fa.booking_source
),
available AS (
    SELECT
        hotel_name,
        room_type,
        COUNT(*) AS room_count
    FROM HOTEL_RESERVATION_DB.CLEAN.CLEAN_ROOMS
    WHERE is_active = TRUE
    GROUP BY hotel_name, room_type
)
SELECT
    ra.summary_date,
    ra.hotel_name,
    ra.room_type,
    ra.total_reservations,
    ra.confirmed_reservations,
    ra.cancelled_reservations,
    ra.checked_out_reservations,
    ra.total_nights_booked,
    ra.total_revenue,
    ra.total_refunded,
    ra.net_revenue,
    ra.avg_stay_duration,
    ra.avg_lead_time_days,
    ROUND(
        (ra.total_nights_booked * 100.0) /
        NULLIF(av.room_count * 30, 0),
    2)                                                  AS occupancy_pct,
    ts.booking_source                                   AS top_booking_source,
    CURRENT_TIMESTAMP()                                 AS aggregated_at
FROM reservation_agg ra
LEFT JOIN top_source ts
    ON ra.hotel_name = ts.hotel_name
    AND ra.room_type = ts.room_type
    AND ts.rn = 1
```

```
LEFT JOIN available av
    ON ra.hotel_name = av.hotel_name
    AND ra.room_type = av.room_type;
```

Step 11: Task Enablement & Manual Trigger

Resumes the task from suspended state, verifies task metadata, and triggers immediate execution.

```
-- Resume Task from suspended state
ALTER TASK HOTEL_RESERVATION_DB.ANALYTICS.TASK_DAILY_RESERVATION_AGGREGATION RESUME;

-- Verify Task Status
SHOW TASKS IN SCHEMA HOTEL_RESERVATION_DB.ANALYTICS;

-- Trigger Immediate Execution for Testing
EXECUTE TASK HOTEL_RESERVATION_DB.ANALYTICS.TASK_DAILY_RESERVATION_AGGREGATION;
```

Step 12: Task History Inspection & Production Monitoring

Inspects Snowflake Information Schema task history to audit execution status, latency, and verified row output.

```
-- Check Task Run History (Last 10 runs)
SELECT
    NAME,
    DATABASE_NAME,
    SCHEMA_NAME,
    STATE,
    SCHEDULED_TIME,
    COMPLETED_TIME,
    ERROR_CODE,
    ERROR_MESSAGE,
    RETURN_VALUE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'TASK_DAILY_RESERVATION_AGGREGATION',
    SCHEDULED_TIME_RANGE_START => DATEADD('DAY', -7, CURRENT_TIMESTAMP()),
    RESULT_LIMIT => 10
))
ORDER BY SCHEDULED_TIME DESC;

-- Overall Execution Duration Audit
SELECT
    NAME,
    STATE,
    SCHEDULED_TIME,
    COMPLETED_TIME,
    DATEDIFF('SECOND', SCHEDULED_TIME, COMPLETED_TIME) AS duration_seconds,
    ERROR_MESSAGE
FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY())
WHERE DATABASE_NAME = 'HOTEL_RESERVATION_DB'
ORDER BY SCHEDULED_TIME DESC
LIMIT 20;
```

```
-- Verify Daily Aggregated Results
SELECT * FROM HOTEL_RESERVATION_DB.ANALYTICS.DAILY_RESERVATION_SUMMARY
ORDER BY hotel_name, room_type;
```

Step 13: Executive Analytical Queries

Suite of executive queries providing overall KPIs, customer loyalty segmentation, channel conversion rates, and monthly revenue trends.

```
-- 1. Overall Portfolio KPIs
SELECT
    COUNT(*)                                                AS total_reservations,
    SUM(CASE WHEN is_cancelled THEN 1 ELSE 0 END)        AS cancellations,
    ROUND(SUM(CASE WHEN is_cancelled THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2)
                                                            AS cancellation_rate_pct,
    SUM(actual_revenue)                                     AS total_revenue,
    ROUND(AVG(stay_duration_days), 1)                       AS avg_stay_nights,
    ROUND(AVG(lead_time_days), 1)                           AS avg_booking_lead_days
FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS;

-- 2. Revenue Contribution by Customer Loyalty Tier
SELECT
    loyalty_tier,
    COUNT(*)              AS bookings,
    SUM(actual_revenue)    AS total_revenue,
    ROUND(AVG(actual_revenue), 2) AS avg_revenue
FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS
WHERE NOT is_cancelled
GROUP BY loyalty_tier
ORDER BY total_revenue DESC;

-- 3. Cancellation Exposure by Booking Channel
SELECT
    booking_source,
    COUNT(*)                                                AS total_bookings,
    SUM(CASE WHEN is_cancelled THEN 1 ELSE 0 END)           AS cancellations,
    ROUND(SUM(CASE WHEN is_cancelled THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2)
                                                            AS cancel_rate_pct,
    SUM(actual_revenue)                                     AS revenue
FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS
GROUP BY booking_source
ORDER BY total_bookings DESC;

-- 4. Monthly Trend Analysis
SELECT
    DATE_TRUNC('MONTH', check_in_date)                      AS month,
    COUNT(*)                                                AS reservations,
    SUM(actual_revenue)                                     AS revenue,
    ROUND(AVG(stay_duration_days), 1)                       AS avg_stay
FROM HOTEL_RESERVATION_DB.ANALYTICS.FACT_RESERVATION_ANALYTICS
WHERE NOT is_cancelled
GROUP BY DATE_TRUNC('MONTH', check_in_date)
```

```
ORDER BY month;
```